

New York Taxi Trip Duration

Project : New York Taxi Trip Duration

Hasselt University

Submitted by:

Ambe Revelation Langsi (2262920)

Anahita Hosseinkhani (2262542)

Elisa Verhofstadt (2261793)

Fidelsia Fri Esibe (2262701)

Presented to:

Prof. Dr. Dirk VALKENBORG

Master of Statistics and Data science

December 15, 2023

1 Introduction

The prediction of a taxi trip duration can be quite challenging due to factors such as weather, passenger demand, location and traffic congestion. In this report, we describe the machine learning-based method used for estimating the duration of taxi trips in New York City.

We participated in the Kaggle competition "New York City Taxi Trip Duration 2017", with the aim of developing a model that could predict trip durations and achieve a low logarithmic root mean squared error (RMSLE).

The Kaggle competition provided both the training and testing data sets of historical taxi trip data from the NYC Taxi and Limousine Commission. The training Data contains trip id, vendor id, the number of passengers, trip distance, geo-coordinates, pickup and drop-off times while the testing dataset excluded the drop-off time (this being the task we want to perform in this project).

We began the project with data exploration, followed by feature engineering (adding more features to the data set which could enhance the models' performance) using some unsupervised machine learning techniques, and experimented with a variety of supervised machine learning algorithms, such as gradient boosting trees and random forest. The evaluation metric for this competition is the Root Mean Squared Logarithmic Error.

The RMSLE is calculated as

$$\epsilon = \sqrt{\frac{1}{n} \sum_{i=1}^n (\log(p_i + 1) - \log(a_i + 1))^2} \quad (1)$$

Where:

ϵ is the RMSLE value (score)

n is the total number of observations in the (public/private) data set,

p_i is our prediction of trip duration, and

a_i is the actual trip duration for i .

$\log(x)$ is the natural logarithm of x .

2 Data Exploration

The training dataset exhibits a shape of (1,458,644, 11), highlighting the absence of any missing values. Additionally, table 1 provides a concise summary of the variables within the dataset.

Table 1: Description of Variables

Variable	Description
id	A unique identifier for each trip
vendor_id	A code indicating the provider associated with the trip record
pickup_datetime	Date and time when the meter was engaged
dropoff_datetime	Date and time when the meter was disengaged
passenger_count	The number of passengers in the vehicle
pickup_longitude	The longitude where the meter was engaged
pickup_latitude	The latitude where the meter was engaged
dropoff_longitude	The longitude where the meter was disengaged
dropoff_latitude	The latitude where the meter was disengaged
store_and_fwd_flag	Flag indicating whether the trip record was stored in vehicle memory before sending to the vendor
trip_duration	Duration of the trip in seconds(target variables)

In order to understand the dataset, it often involves visualizing the distributions of features and their relationships. These visual representations not only clarify the data but frequently spark fresh avenues of investigation.

The following are some insights from the data:

The maximum number of passengers is 9, while a minimum of 0 passengers is observed, which doesn't seem to make sense. This could be due to erroneous input data or trip cancellations by passengers. Predominantly, the trips are undertaken by either one or two passengers.

There are two vendor IDs and the second service provider is the most preferred choice among New Yorkers. The store and forward flag indicates whether the trip record had been stored in the vehicle's memory before transmission to the vendor, typically owing to the vehicle lacking a connection to the server. With 'Y' representing store and forward and 'N' indicating otherwise, the data distinctly illustrated that less than 1% of trips had been cached in memory before transmission.

Furthermore, it can be concluded from figure 1 that the randomness observed in the train and test data sets in this scenario presents an advantageous aspect for our analysis. This randomness signifies that the data distribution across both sets doesn't exhibit any discernible pattern or systematic bias, enabling us to leverage unsupervised learning techniques effectively.

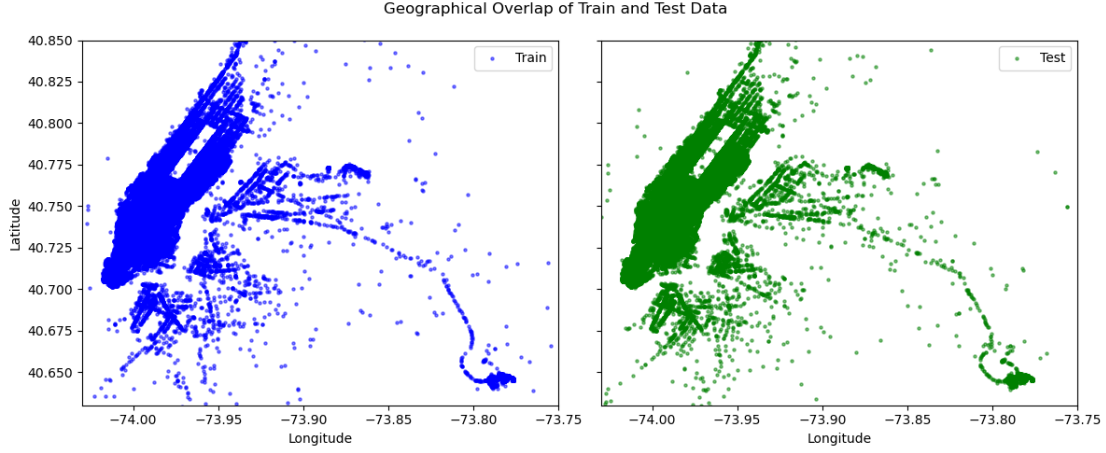


Figure 1: Mapping of pickup points. Left: latitude and longitude coordinates of the train dataset. Right: latitude and longitude coordinates of the test dataset.

The Date and time contains valuable information that needs to be extracted as separate features. According to Figure 2, which shows the number of trips for each weekday, it is observed that Fridays and Saturdays are the preferred days for New Yorkers to explore the city.

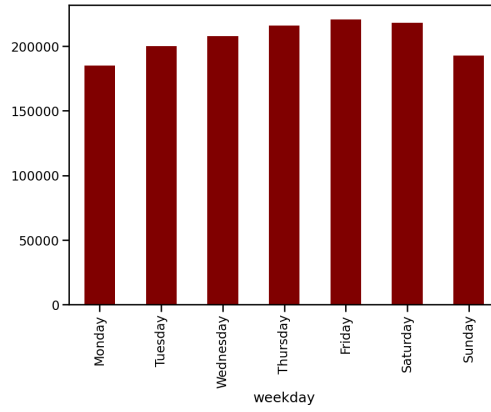


Figure 2: Number of trips every week.

In Figure 3a, distinct trends in trip volumes are evident, revealing noteworthy variations between months and days of the week. January and June exhibit fewer trips compared to the busier months of March and April, potentially attributable to events or seasonal changes. Additionally, Figure 3b indicates that from February onward, the duration of trips increase linearly until May, where there is a slight change.

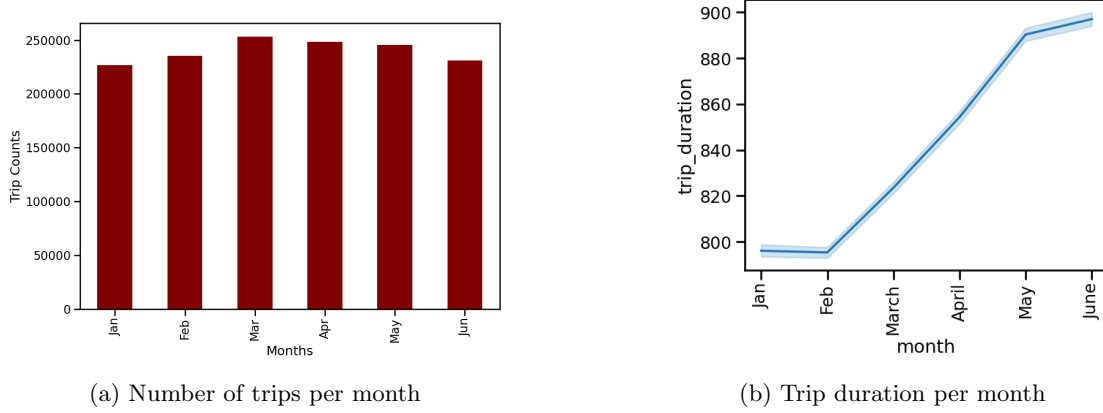


Figure 3: Exploration for Month parameter

According to Figure 4, trips are quietest around 5 am but surge between 5 pm and 8 pm, which are the peak activity hours. On weekends, especially early mornings on Saturdays and Sundays, along with Friday mornings, we see increased trip durations, hinting lively social activities or nightlife. Interestingly, these weekends show fewer trips during typical weekday rush hours. On sunday evenings, we see a gradual decrease in trips, indicating the winding down of weekend activities before the weekday begins. These patterns unveil dynamic rhythms in weekly travel behaviors.

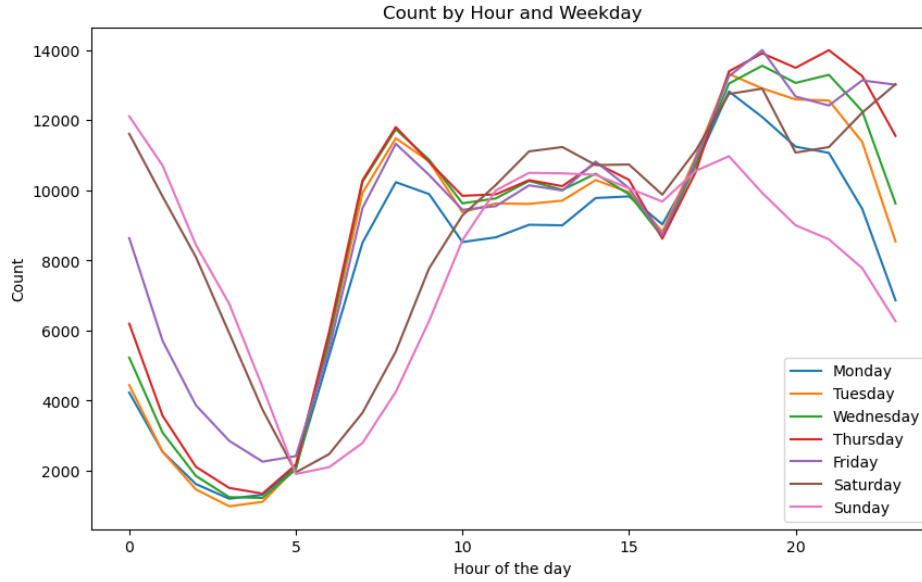


Figure 4: Number of trips per hours based on week day.

3 Feature Engineering

In preparation for predicting taxi trip durations, the initial dataset is extended through incorporation of multiple new features derived from the existing dataset's provided information. These new features were strategically chosen to encompass a wide range of factors that could potentially impact the trip duration of a taxi in New York City. The inclusion of these more informative features will therefore enhance the model's precision.

Our current set of features emerged from the combination of several approaches. First of all, basic calculations and manipulations based on variables of the original dataset (e.g., formulas, binary indicators based on coordinates, ...) were used to create new features. Secondly, external datasets were integrated and aligned with the observations in the original dataset. Next, principal component analysis (PCA) was used to transform latitude and longitude points to two corresponding PCA components in order to disentangle the correlation between the two values and make them better suitable for models based on decision trees. Lastly, the coordinates of pickup and drop-off points were once more transformed by forming groups of closely related points through clustering techniques.

The following sections provide an overview of the additional features integrated into the dataset.

3.1 Features Based on Transformation of Original Dataset Variables

Year Year as specified in the pickup time.

Weekday Day of the week as specified in the pickup time.

Hour Hour of the day as specified in the pickup time.

Month Month of the year as specified in the pickup time.

Distance miles Distance measured in miles between the pickup and drop-off point in a straight line. The Haversine formula is used to transform the latitude and longitude points of the two locations to the distance measure:

$$d = 2r \arcsin \left(\sqrt{\sin^2 \left(\frac{\phi_1 - \phi_2}{2} \right) + \cos(\phi_1) \cos(\phi_2) \sin^2 \left(\frac{\lambda_1 - \lambda_2}{2} \right)} \right)$$

where:

ϕ_1, ϕ_2 : latitudes of points 1 and 2 in radians

λ_1, λ_2 : longitude of points 1 and 2 in radians

r : Earth's radius in miles

Direction The direction from the pickup point to the drop-off point, denoted by their angle measured clockwise from -180° to 180° .

Airport Binary indicator of whether either the pickup or drop-off location is near one of the three major airports in New York City: John F. Kennedy International Airport, Newark Liberty International Airport, or LaGuardia Airport¹.

¹Coordinates used to identify square of airport location are, JFK: (73°50'W, 40°40'N; 73°45'W, 40°40'N; 73°45'W, 40°37'N; 73°50'W, 40°37'N), NWR: (74°11'W, 40°425'N; 74°9'W, 40°425'N; 74°9'W, 40°40'N; 74°113'W, 40°40'N), LGA (73°533'W, 40°475'N; 73°51'W, 40°475'N; 73°51'W, 40°46'N; 73°533'N, 40°46'N)

Pickup borough District of New York City based on the pickup longitude and latitude. The possible values for the boroughs are: the Bronx, Brooklyn, Manhattan, Queens, Staten Island, and Outside NYC.

Dropoff borough District of New York City based on the drop-off longitude and latitude. The possible values for the boroughs are: the Bronx, Brooklyn, Manhattan, Queens, Staten Island, and Outside NYC.

3.2 Features Based on External Datasets

3.2.1 Historical Hourly Weather Data from 2012-2017

The dataset from which the features listed below are extracted describes the hourly weather data from 2012 to 2017 ². New York City is one of the 30 US cities included in this dataset. Therefore, based on the date and hour of the pickup time, individual taxi trips are matched with weather conditions at that moment.

Humidity % of humidity at the moment of the taxi trip.

Temperature Temperature in degrees Kelvin at the moment of the taxi trip.

Weather description Description of the weather conditions at the moment of the taxi trip. Possible values for this variable are: broken clouds, drizzle, few clouds, fog, haze, heavy intensity rain, heavy snow, light intensity drizzle, light rain, light snow, mist, moderate rain, overcast clouds, proximity thunderstorm, scattered clouds, sky is clear, smoke, snow, thunderstorm, thunderstorm with light drizzle, and thunderstorm with light rain.

Wind direction Wind direction in meteorological degrees at the moment of the taxi trip.

Wind speed Wind speed in m/s at the moment of the taxi trip.

3.2.2 OSRM Data

The Open Source Routing Machine (OSRM) uses data from the OpenStreetMap project in order to find the fastest route between two points. This can be used to provide routes between taxi pickup and drop-off locations ³, from which additional information can be extracted about the taxi trip.

Total distance Distance in meter of the OSRM route between pickup and drop-off point.

Total travel time Time in seconds of the OSRM route between pickup and drop-off point.

Number of steps The number of distinct steps (e.g., turn left, cross intersection,...) a vehicle follows based on the OSRM route

3.3 PCA

PCA was used for the purpose of standardizing latitude and longitude coordinates of the pickup and drop-off points in the final feature dataset. The first two PCA components were used based

²Weather dataset can be found at: <https://www.kaggle.com/datasets/selfishgene/historical-hourly-weather-data/data>

³The dataset with the fastest routes can be found at: <https://www.kaggle.com/datasets/evgeniypolin/osrmdatatrain>

on the pickup or drop-off points. This results in the projection of the original points on a new coordinate space, as the points now are a linear combination of the original latitude and longitude measures. As can be seen in Figure 5, this procedure keeps intact the relationship among the points, as no dimension reduction is required, only resulting in rotation of the coordinate space of the points.

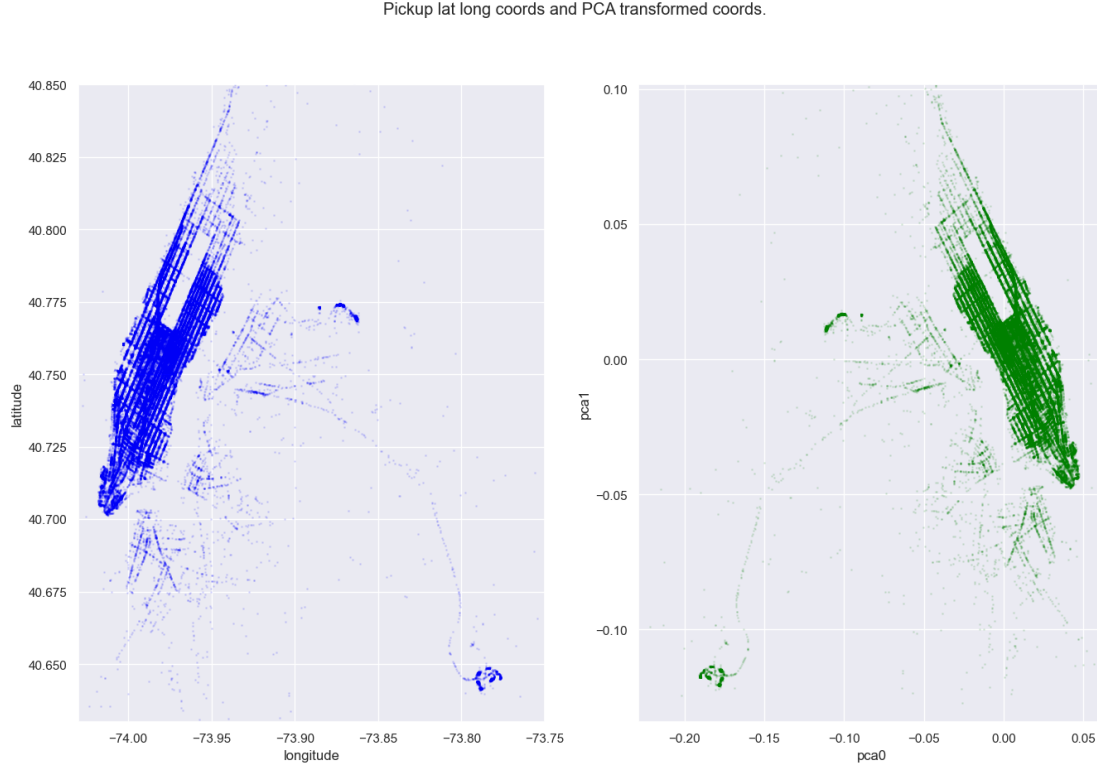


Figure 5: Mapping of pickup points. Left: latitude and longitude coordinates of the original dataset. Right: first and second principal component for the PCA transformation.

PCA 0 - pickup First component resulting from PCA of latitude and longitude coordinates of pickup points. This represents a linear combination of these coordinates.

PCA 1 - pickup Second component resulting from PCA of latitude and longitude coordinates of pickup points. This represents a linear combination of these coordinates.

PCA 0 - drop-off First component resulting from PCA of latitude and longitude coordinates of drop-off points. This represents a linear combination of these coordinates.

PCA 1 - drop-off Second component resulting from PCA of latitude and longitude coordinates of drop-off points. This represents a linear combination of these coordinates.

3.4 Clustering

Clustering of pickup and drop-off points aimed to identify and group points close in proximity. A K-means technique with as input latitude and longitude points of both pickup and drop-off

points was used to construct these groups. The group size was set at 100. The result of this procedure for pickup points is visualized in Figure 6.

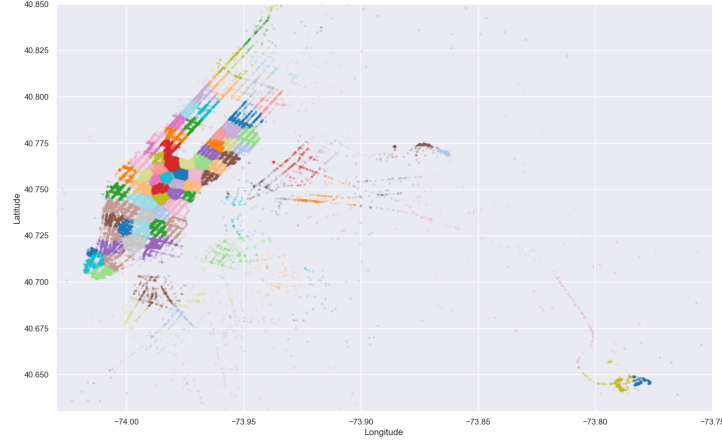


Figure 6: 100 different pickup point clusters identified based on latitude and longitude coordinates.

4 Modelling

Two supervised machine learning techniques were used in this project: random forest and the gradient boosting. The XGboost model gave a lower validation RMSLE as compared to the random forest, and as such, the results of the XGboost model will be reported.

The log-transformed trip duration was used to fit an XGBoost model. The transformation was done so as to minimise the effect of outliers in the test data set that are also present in the data set and also to ease the calculation of our test RMSLE. The complete training data set was split into 20 (test) and 80 (train) data sets and used to fit and test the model.

4.1 Features Used

We used a total of 28 features to do the taxi trip duration prediction in New York City.

'passenger_count', 'pickup_longitude', 'pickup_latitude', 'dropoff_longitude', 'dropoff_latitude', 'store_and_fwd_flag', 'pickup_borough', 'dropoff_borough', 'year', 'weekday', 'hour', 'month', 'distance_miles', 'direction', 'Airport', 'humidity', 'temperature', 'weather description', 'wind direction', 'wind speed', 'total_distance', 'total_travel_time', 'number_of_steps', 'pickup_pca0', 'pickup_pca1', 'dropoff_pca0', 'dropoff_pca1', 'pickup_cluster', 'dropoff_cluster'

4.2 Extreme Gradient Boosting(XGBoost)

After fine-tuning the parameters of this model, our best test RMSLE was 0.39101. Below, we present the feature importance score of our model.

Feature Importance	
dropoff_pca1	2692.0
pickup_latitude	2566.0
distance_miles	2543.0
pickup_longitude	2421.0
direction	2386.0
total_travel_time	2362.0
pickup_pca1	2338.0
dropoff_longitude	2105.0
temperature	2099.0
dropoff_pca0	1935.0
pickup_pca0	1869.0
total_distance	1809.0
dropoff_latitude	1772.0
wind_direction	1570.0
dropoff_cluster	1391.0
hour	1218.0
pickup_cluster	1009.0
number_of_steps	927.0
weather_description	775.0
humidity	772.0
wind_speed	636.0
weekday	584.0
month	557.0
passenger_count	216.0
dropoff_borough	185.0
pickup_borough	181.0
Airport	49.0
store_and_fwd_flag	8.0

5 Submission and score on Kaggle

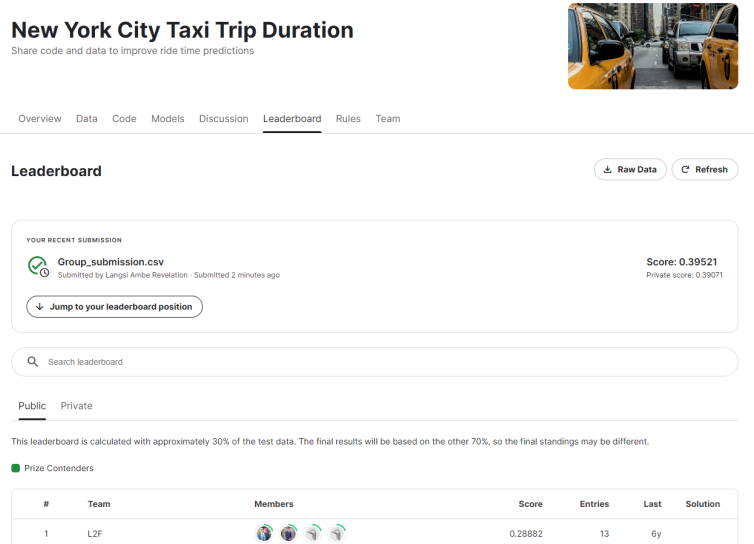


Figure 7: Kaggle Submission Score

New York City Taxi Trip Duration [Late Submission](#) ...

Overview Data Code Models Discussion **Leaderboard** Rules Team

#	Team	Members	Score	Entries	Last	Solution
407	PiyushSupe		0.39450	6	6y	
408	AngiWang		0.39450	4	6y	
409	er9213		0.39450	1	6y	
410	fire_noob		0.39450	1	6y	
411	razerx		0.39468	1	6y	
412	Dmitry Storozhenko		0.39478	5	6y	
413	tiramisu		0.39487	5	6y	
414	InsanePro		0.39490	6	6y	
415	lalala		0.39516	5	6y	
416	demon		0.39521	6	6y	
417	CarlesBalsach		0.39521	6	6y	
418	avpopov		0.39529	19	6y	
419	Oleg Panichev		0.39541	7	6y	
420	YusukeOi		0.39541	4	6y	
421	Vlad Nik		0.39547	8	6y	
422	tanzaku		0.39552	22	6y	

Figure 8: Kaggle Position on the Public Leaderboard

Our submission score was 0.39521 which put us at position 416 out of 1254 competitors.

6 References

- ISLP (Introduction to Statistical Learning in Python) textbook by Trevor Hastie, Gareth James, Daniela Witten, Robert Tibshirani, and Jonathan Taylor is available at: https://hastie.su.domains/ISLP/ISLP_website.pdf.download.html
- The analysis presented in the Kaggle kernel by Gábor Fodor: <https://www.kaggle.com/code/gaborfodor/from-eda-to-the-top-1b-0-367>

7 Appendix

Important Codes

```
{Libraries}
import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
import matplotlib
matplotlib.rcParams['figure.figsize']=(10,18)
%matplotlib inline
from datetime import datetime
from datetime import date
import xgboost as xgb
from sklearn.cluster import MiniBatchKMeans
import seaborn as sns # plot beautiful charts
import holidays
import warnings
from sklearn.model_selection import train_test_split
from sklearn.decomposition import PCA
sns.set()
warnings.filterwarnings('ignore')
from sklearn.ensemble import RandomForestRegressor,
    RandomForestClassifier

{Read data}
data = pd.read_csv('train_no_filter.csv', parse_dates=['pickup_
    datetime', 'dropoff_datetime'])
test= pd.read_csv('test_no_filter.csv', parse_dates=['pickup_
    datetime'])

{PCA}
plt.rcParams['figure.figsize'] = [16, 10]
np.random.seed(1987)
N = 100000 # number of sample rows in plots
coords = np.vstack((data[['pickup_latitude', 'pickup_longitude']].
    values,
                    data[['dropoff_latitude', 'dropoff_longitude'
    ]].values,
                    test[['pickup_latitude', 'pickup_longitude']].
    values,
                    test[['dropoff_latitude', 'dropoff_longitude'
    ]].values))
pca = PCA().fit(coords)
data['pickup_pca0'] = pca.transform(data[['pickup_latitude', '
    pickup_longitude']])[:, 0]
data['pickup_pca1'] = pca.transform(data[['pickup_latitude', '
    pickup_longitude']])[:, 1]
```

```
data['dropoff_pca0'] = pca.transform(data[['dropoff_latitude', '
dropoff_longitude']])[:, 0]
data['dropoff_pca1'] = pca.transform(data[['dropoff_latitude', '
dropoff_longitude']])[:, 1]
test['pickup_pca0'] = pca.transform(test[['pickup_latitude', '
pickup_longitude']])[:, 0]
test['pickup_pca1'] = pca.transform(test[['pickup_latitude', '
pickup_longitude']])[:, 1]
test['dropoff_pca0'] = pca.transform(test[['dropoff_latitude', '
dropoff_longitude']])[:, 0]
test['dropoff_pca1'] = pca.transform(test[['dropoff_latitude', '
dropoff_longitude']])[:, 1]
city_long_border = (-74.03, -73.75)
city_lat_border = (40.63, 40.85)

fig, ax = plt.subplots(ncols=2)
ax[0].scatter(data['pickup_longitude'].values[:N], data['pickup_
latitude'].values[:N],
              color='blue', s=1, alpha=0.1)
ax[1].scatter(data['pickup_pca0'].values[:N], data['pickup_pca1'].
values[:N],
              color='brown', s=1, alpha=0.1)
fig.suptitle('Pickup_lat_long_coords_and_PCA_transformed_coords.')
ax[0].set_ylabel('latitude')
ax[0].set_xlabel('longitude')
ax[1].set_xlabel('pca0')
ax[1].set_ylabel('pca1')
ax[0].set_xlim(city_long_border)
ax[0].set_ylim(city_lat_border)
pca_borders = pca.transform([[x, y] for x in city_lat_border for y
in city_long_border])
ax[1].set_xlim(pca_borders[:, 0].min(), pca_borders[:, 0].max())
ax[1].set_ylim(pca_borders[:, 1].min(), pca_borders[:, 1].max())
plt.show()

{clustering}

sample_ind = np.random.permutation(len(coords))[:500000]
kmeans = MiniBatchKMeans(n_clusters=100, batch_size=10000).fit(
    coords[sample_ind])
data.loc[:, 'pickup_cluster'] = kmeans.predict(data[['pickup_
latitude', 'pickup_longitude']])
data.loc[:, 'dropoff_cluster'] = kmeans.predict(data[['dropoff_
latitude', 'dropoff_longitude']])
```

```
test.loc[:, 'pickup_cluster'] = kmeans.predict(test[['pickup_
    latitude', 'pickup_longitude']])
test.loc[:, 'dropoff_cluster'] = kmeans.predict(test[['dropoff_
    latitude', 'dropoff_longitude']])

fig, ax = plt.subplots(ncols=1, nrows=1)
ax.scatter(data.pickup_longitude.values[:N], data.pickup_latitude.
    values[:N], s=10, lw=0,
            c=data.pickup_cluster[:N].values, cmap='tab20', alpha
            =0.2)
ax.set_xlim(city_long_border)
ax.set_ylim(city_lat_border)
ax.set_xlabel('Longitude')
ax.set_ylabel('Latitude')
plt.show()

{XGBOOST MODEL}
model_features1 = ['passenger_count', 'pickup_longitude', 'pickup_
    latitude',
                    'dropoff_longitude', 'dropoff_latitude', 'store_and_fwd_
    flag',
                    'pickup_borough', 'dropoff_borough', 'year', 'weekday',
                    'hour', 'month', 'distance_miles', 'direction',
                    'Airport', 'humidity', 'temperature',
                    'weather_description', 'wind_direction', 'wind_speed', '
    total_distance',
                    'total_travel_time', 'number_of_steps', 'pickup_pca0', '
    pickup_pca1', 'dropoff_pca0', 'dropoff_pca1',
                    'pickup_cluster', 'dropoff_cluster']

y1=data['log_trip_duration']
Xtr, Xv, ytr, yv = train_test_split(data[model_features1].values,
    y1, test_size=0.2, random_state=1987)
dtrain = xgb.DMatrix(Xtr, label=ytr)
dvalid = xgb.DMatrix(Xv, label=yv)
dtest = xgb.DMatrix(test[model_features1].values)
watchlist = [(dtrain, 'train'), (dvalid, 'valid')]

# Try different parameters)
xgb_pars = {'min_child_weight': 50, 'eta': 0.3, 'colsample_bytree'
    : 0.3, 'max_depth': 10,
            'subsample': 0.8, 'lambda': 1., 'nthread': 4, 'booster
    ' : 'gbtree', 'silent': 1,
            'eval_metric': 'rmse', 'objective': 'reg:linear'}
num_epochs = 150
early_stopping_rounds = 50
```

```

model = xgb.train(xgb_pars, dtrain, num_epochs, watchlist, early_
    stopping_rounds=early_stopping_rounds,
                    maximize=False, verbose_eval=10)

{Submission file}

ytest = model.predict(dtest)
print('Test_shape_OK.') if test.shape[0] == ytest.shape[0] else
    print('Oops')
test['trip_duration'] = np.exp(ytest) - 1
test[['id', 'trip_duration']].to_csv('Revelation_submission.csv.gz',
    index=False, compression='gzip')

{Feature importance}
dtrain = xgb.DMatrix(Xtr, label=ytr, feature_names=model_features1
    )
dvalid = xgb.DMatrix(Xv, label=yv, feature_names=model_features1)
dtest = xgb.DMatrix(test[model_features1].values)

# Get feature importance
feature_importance = model.get_score(importance_type='weight')

# Map feature indices to feature names
feature_importance_named = {model_features1[int(key[1:]):]: value
    for key, value in feature_importance.items()}

# Print feature importance
print("Feature_Importance:")
for feature, importance in sorted(feature_importance_named.items(),
    key=lambda x: x[1], reverse=True):
    print(f"{feature}:_{importance}")

# Plot feature importance
xgb.plot_importance(model)
plt.show()

```